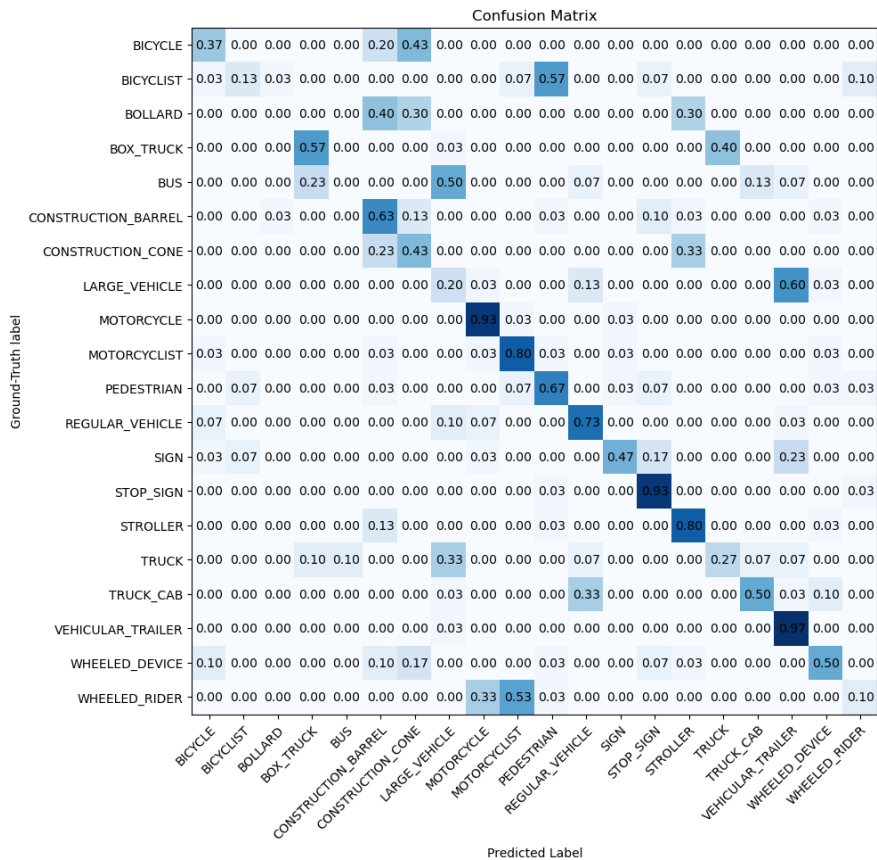


Part 2: Voxel Baseline Model

insert confusion matrix for baseline model on test dataset



Part 2: Voxel Baseline Model

What are the top confusion pairs in the matrix and why might this be the case?

The main confusion pairs are regular vehicle/large vehicle and wheeled rider/bicyclist. These classes have pretty similar spatial footprints in a 4x4x4m grid. Large vehicles fill many of the voxels in similar rectangular patterns, while the bicyclists are small and sparse, so the coarse 1m³ voxel resolution can't capture the geometric differences between these pairs.

To which classes would adding finer geometry help the most and why?

The classes sign, pedestrian, bicyclists, and wheeled rider would be helped by finer geometry. These are the small objects whose shapes like thin poles are lost at 1m³ resolution. A finer voxel grid would preserve the structural difference that coarse voxels merge into one cell.

Part 2: Voxel Baseline Model

Which classes does this baseline perform well for? Why?

The baseline works best for large distinct classes like bus, bollard, and construction cone. Since these objects have a unique voxel occupancy pattern – a bus filling many voxels in a large shape, and a bollard filling only a few voxels in a narrow column, it makes them separable even with coarse voxel features,

What are some ways we can improve this voxel based model?

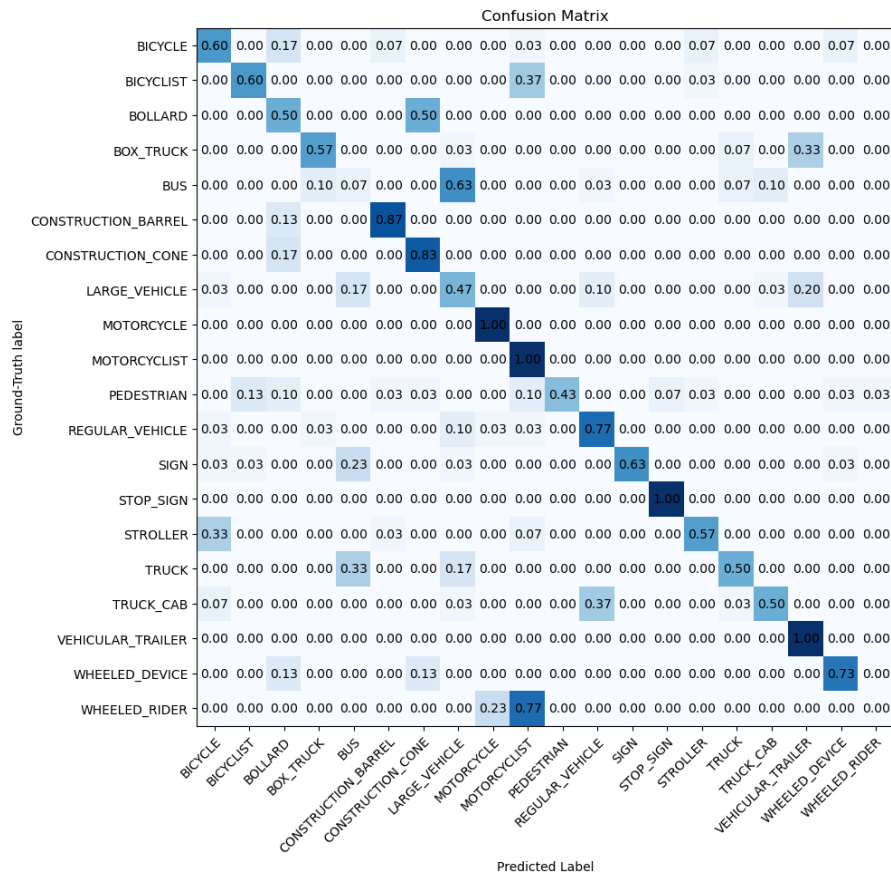
Improvements could be with things like a finer voxel resolution like 8x8x8 or 16x16x16 to get more geometric detail. Also, a multi resolution voxel grid where coarse and fine features are combined, or replacing the single linear classifier with a deeper MLP.

Which mode (count or occupancy) achieves higher accuracy? Why do you think this is the case?

Occupancy mode has a higher accuracy – 50%. Since count mode preserves point density, it's sensitive to the changing number of lidar returns per object. Objects further from the sensor have fewer returns, making the count features inconsistent for the same class. Binary occupancy makes this variation normalized and gives a stable return across instances.

Part 3: Simplified PointNet

Insert confusion matrix for PointNet model on test dataset



Part 3: Simplified PointNet

Which classes does this model perform well for? Why?

PointNet does well for geometrically distinct classes like bollard, sign, construction cone and regular vehicle. These classes have unique point cloud shapes – bollards with their thin vertical cylinders, signs with flat rectangular surfaces, and vehicles with box like structures. PointNet's per-point MLP gets these local geometric features more effectively than voxel counts.

Which classes are most misclassified? What were they classified as and why do you think this is the case?

The most misclassified classes are wheeled rider, bus, large vehicle, and pedestrian. Wheeled rider is confused with bicyclist and motorcyclist because of its similar point patterns. Bus is misclassified with large vehicle and truck since all large vehicles have dense box shaped clouds. Pedestrians is confused with bicyclists since they have similar heights and point density.

Part 3: Simplified PointNet

What changes in the architecture or training process can be done to improve performance?

Architectural improvements could be wider hidden dimensions, adding dropout to the classifier for regularization, and using the T-Net input transform for rotation invariance. Training improvements would be things like training for more epochs, using a cosine learning rate scheduler, adding weight decay, or applying data augmentation like random point jitter.

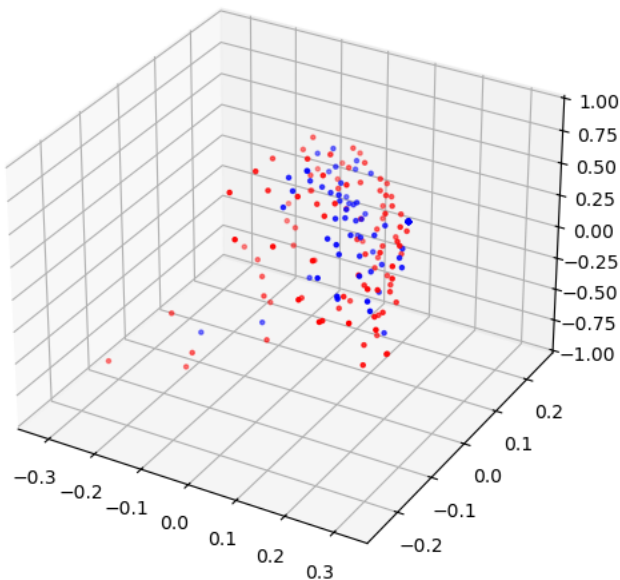
Did the top confusion pair in the matrix remain the same as in Part 2? Why or why not?

The top confusion pairs somewhat changed. In Part 2, regular vehicle/large vehicle was the main confusion because they had similar voxel occupancy. PointNet eased this confusion since it captures per-point geometry. However, bus/large vehicle and wheeled rider/bicyclist are still confused because these classes are geometrically similar at the individual point level, and no amount of per-point processing can fully fix this without having more context.

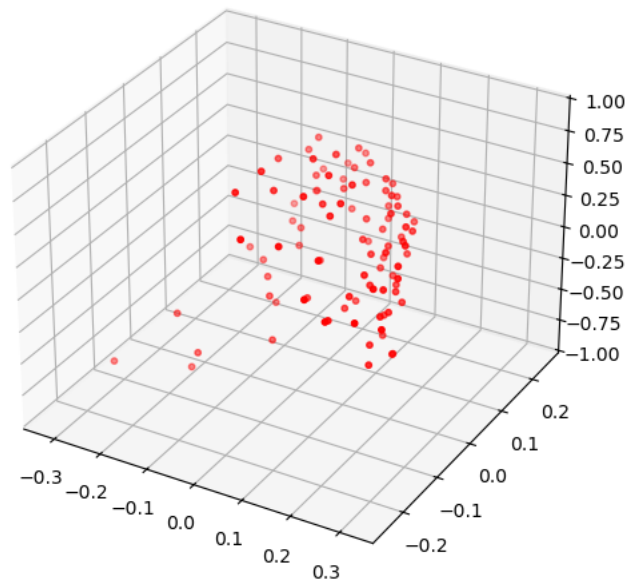
Part 4: Analysis

insert visualization of critical points on the PEDESTRIAN/11.txt point cloud

PEDESTRIAN - All Points (red=critical)



PEDESTRIAN - Critical Points Only

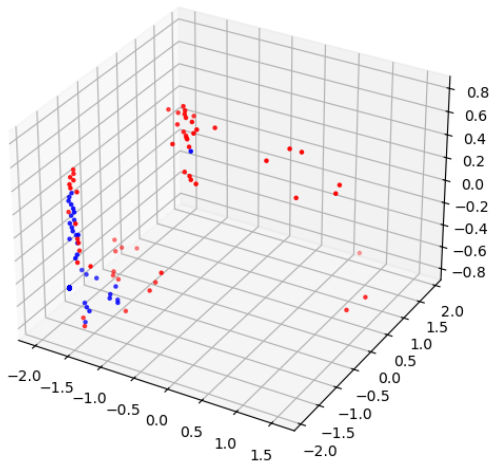


Part 4: Analysis

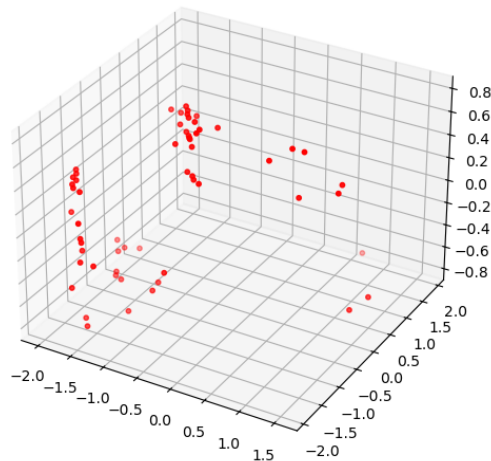
insert visualization critical points on one other point cloud, why would it make sense for these to be selected as critical points?

For regular vehicle, the critical points are along the roof edges, corners, and side mirrors of the vehicle. These edge and boundary points are the most distinctive features of a vehicle because they show the overall bounding shape and dimensions. It makes sense for the model to pick these points since they have the most discriminative information about the object's class, and interior points on flat surfaces are less informative.

REGULAR_VEHICLE - All Points (red=critical)



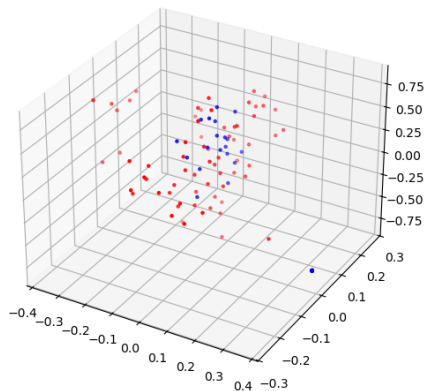
REGULAR_VEHICLE - Critical Points Only



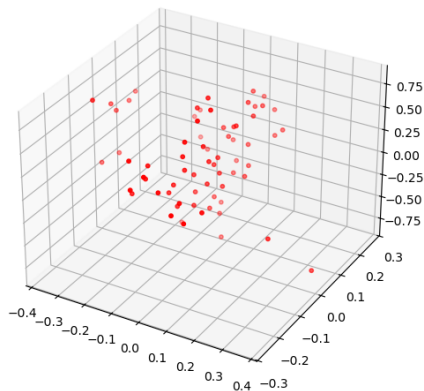
Part 4: Analysis

Plot critical points for two more classes which are misclassified

WHEELED_RIDER - All Points (red=critical)



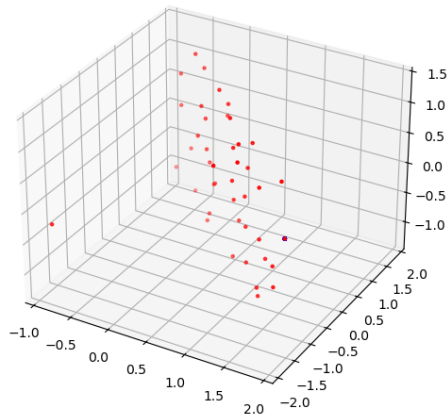
WHEELED_RIDER - Critical Points Only



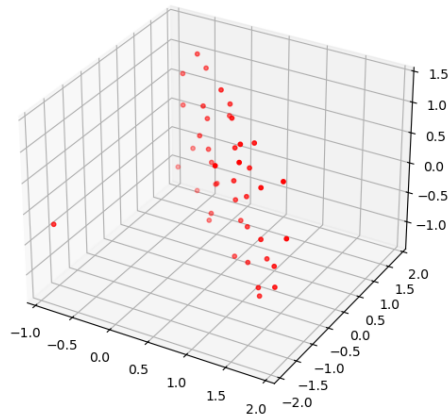
Wheeled rider: Critical points are sparse and scattered with no clear pattern, which is why it gets confused with bicyclist, since both are small upright sparse clouds with no distinctive anchor points.

BUS: Critical points cluster on large flat side surfaces shared with large vehicle and truck. Since all these big vehicles fill the box similarly, the model can't find features unique to buses.

BUS - All Points (red=critical)



BUS - Critical Points Only



Part 5: PointNet + T-Net (Extra Credit)

What is the motivation behind using T-Net in PointNet?

PointNet's encoder is an MLP with max pooling, which doesn't have a built-in invariance to spatial transformations. Objects of the same class show up in different orientations depending on where they are relative to the sensor. T-Net fixes this by learning a 3×3 transformation matrix from the input point cloud and applying it to change the orientation before classification.

Which classes saw the most improvement from including T-Net? Why do you think this is the case?

Classes that benefit the most from T-Net are those whose point clouds vary in orientation, like pedestrian, wheeled rider, and bicyclists. These objects can face any direction relative to the sensor, so their point clouds do vary in orientation. T-Net aligns them before PointNet processes them, reducing the variance the classifier has to handle. Large symmetric objects like bus or bollard don't benefit as much because their point clouds look similar regardless of the rotation.